

JWT

JSON Web Tokens

Autenticación de Aplicaciones

Las nuevas tendencias, en cuanto al desarrollo web, es la autenticación en aplicaciones web por medio de tokens y que el *backend* sea un API RESTful *stateless*.

JSON Web Token (JWT) es un estándar abierto (RFC-7519) basado en JSON para crear un token que sirva para enviar datos entre aplicaciones o servicios y garantizar que sean válidos y seguros. Se trata de una tecnología desarrollada por la empresa **Auth0** cuyo objetivo es la transmisión segura de información a través de una red insegura (Internet).

Los JSON Web Tokens (JWT) se han convertido en un estándar en la autenticación de aplicaciones, permiten de una forma simple, **identificarse en el servidor mediante un token**.

El token es generado por el servidor y transmitido al cliente, que deberá enviarlo en cada petición al servidor para poder ser autenticado.

¿Qué es un token JWT?

Un token es una cadena alfanumérica que tiene tres partes codificadas en Base64, cada una de ellas separadas por un punto.

Un token JWT, sería una cadena de este tipo:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJmMONTY3ODkwlwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iOnRyZWV9.TjVA95OrM7E2cBab30RMHrHDcEfxjoYZgeFONFh7HgQ

La cadena está dividida en 3 secciones separadas por un punto. Estas tres secciones tienen un significado:

1. **HEADER:** la primera parte, corresponde al Header, y en él se almacena: el tipo de token y el algoritmo de encriptación. El algoritmo elegido se utilizará para realizar la codificación y la posterior verificación.

Nota: Los algoritmos de cifrado más utilizados son HMAC con SHA-256 (HS256) código de autenticación de mensajes en clave-hash o código de autenticación de mensaje basado en hash, es una construcción específica para calcular un código de autenticación de mensaje (MAC) que implica una función **hash** criptográfica en combinación con una **llave criptográfica secreta**. Pueden utilizarse algoritmos RSA y ES256 (clave pública y clave privada).

2. **PAYLOAD (carga útil):** la segunda parte, contiene los datos que identifican al usuario, como puede ser su ID, nombre de usuario, fecha de creación del token, fecha de expiración del token, etc. El payload de un JWT es un string JSON que puede tener cualquier propiedad, existen varias propiedades definidas en el estándar.
3. **FIRMA:** La tercera parte es la firma digital, la cual se genera con las secciones anteriores y sirve para validar que el contenido no haya sido alterado. Se codifica en Base64 y después se encripta con una clave que se guarda en el servidor.

Ejemplo del Header

```
{
  "alg": "ES256", //Algoritmo de encriptación elegido
  "typ": "JWT"
}
```

Ejemplo del Payload

```
{
  "iat":time(), //fecha de creación del token
  "exp" : time() + (60 * 60 * 24), //fecha de expiración del token (1 día)
  "id": "1",
  "email": "promotor@gmail.com",
  "rol": "promotor"
}
```

Ejemplo del Firma:

TJVA95OrM7E2cBab30RMHrHDCEfXjoYZgeFONFh7HgQ

Firma de un token JWT

La firma se compone del header y el payload codificados en base64 y después encriptados, con un algoritmo elegido.

Una firma que nos permite verificar si el token es válido, y aquí es donde radica el quid de la cuestión, ya que si estamos tratando de hacer una comunicación segura entre dos partes y hemos visto que podemos coger cualquier token y ver su contenido con una herramienta sencilla, ¿dónde reside entonces la potencia de todo esto?

- **En la palabra secreta**, elegida en por la aplicación y guardada en el servidor.
- **En la clave privada almacenada por el propietario y que nunca se expone.**

La firma se construye de tal forma que vamos a poder verificar que el remitente es quien dice ser, y que el mensaje no se ha modificado por el camino.

Comparativa de Algoritmos

Algoritmo	Tipo	Seguridad	Usos ideales
HS256	Simétrico	Moderada, clave secreta compartida	Aplicaciones simples.
HS512	Simétrico	Alta, con clave secreta compartida	Requiere compatibilidad con SHA-512.
RS256	Asimétrico	Muy alta, clave pública y privada	Sistemas distribuidos.
ES256	Asimétrico	Muy alta y eficiente, clave pública y privada	Sistemas con limitaciones de recursos.

Algoritmo ES256

ES256 es un **algoritmo asimétrico** de firma digital utilizado en JSON Web Tokens (JWT)

Un algoritmo asimétrico o criptografía de clave pública, es un método de cifrado que utiliza dos claves diferentes pero matemáticamente relacionadas: una clave pública y una clave privada. Esta característica fundamental distingue a los algoritmos asimétricos de los simétricos, que usan una única clave compartida.

Características:

1. **Par de claves:** Cada usuario o entidad tiene un par de claves:
 - Una clave pública que se puede compartir libremente.
 - Una clave privada que debe mantenerse en secreto.
2. **Funciones unidireccionales:** Las claves están relacionadas matemáticamente, pero es computacionalmente inviable derivar la clave privada a partir de la pública.

En el contexto de JWT, **la firma se realiza siempre con la clave privada**, mientras que la verificación se hace con la clave pública correspondiente.

- La clave privada se usa para firmar el token, garantizando su autenticidad e integridad.
- La clave pública se utiliza para verificar la firma, permitiendo a cualquier parte con acceso a esta clave validar el token sin comprometer la seguridad del sistema.

¿Cómo funciona el algoritmo ES256?

1. Firma con clave privada:

- El propietario de los datos utiliza su clave privada para firmar el contenido, el **JWT** (JSON Web Token).
- La firma se genera a partir del contenido y asegura que este no se puede modificar sin invalidarla.

2. Verificación con clave pública:

- Cualquiera puede usar la clave pública correspondiente para verificar que:
 - El contenido no ha sido modificado.
 - La firma fue generada por el propietario de la clave privada.

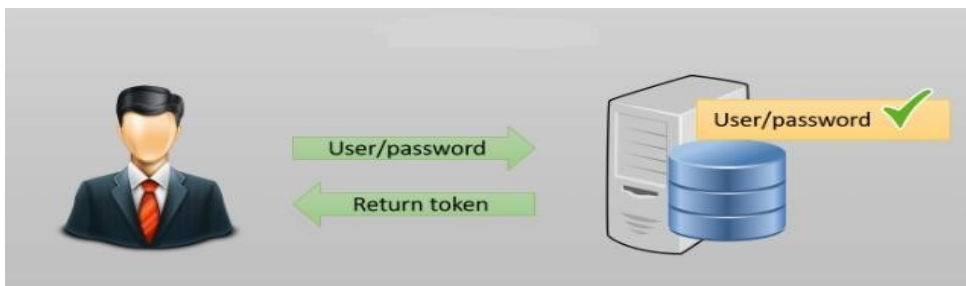
¿Cómo se utiliza los JWT?

Los JWT se utilizan para autenticar a los usuarios en una aplicación web, para ello, el usuario requiere de un login tradicional como es el **usuario y password**.

Una vez, que el servidor valida que el usuario y contraseña son correctos, le envía un token al usuario.

Este token **deberá guardarlo el cliente**, pues de aquí en adelante, todas las peticiones que realice al servidor, deberá llevar el token adjunto.

El token es por lo general almacenado en Cookies o en el LocalStorage del navegador, y cuando es necesario enviar una petición al servidor, se recupera y se envía en una cabecera(header).



Token JWT seguro

- Aunque el algoritmo nos permita verificar la firma, y, por tanto, confiar o no, **tanto el encabezado como el cuerpo si llevan muchos datos van en abierto**, ya que Base64 no es un cifrado, es simplemente una codificación que es muy fácilmente decodificable.
- JWT recomienda siempre que la comunicación entre partes **se realice con HTTPS** para encriptar el tráfico y así añadir más seguridad.
- **El token no requiere que el servidor lo almacene** para compararlos cuando lo envíe el cliente, pues el token, por si solo, puede ser **auto validado**, y como tiene un payload, es posible determinar de quien es el token.

Por ejemplo, en el payload podemos enviar los datos que necesitemos del usuario:

```
$payload = [  
    'iat' => $horaActual, // Hora en la que se emitió el token  
    'exp' => $tiempoExpira, // Tiempo en el que expira el token  
    'idusuario' => 1,  
    'email' => 'promotor@gmail.com',  
    'rol' => 'promotor'  
];
```

Cuando el servidor desencripte el token, podrá recuperar este payload y con ello, podrá saber qué usuario es, e incluso, es posible guardar datos adicionales como el rol o cualquier dato que necesitemos en formato JSON.

- **El token expira**, es decir, **el token solo es válido por un tiempo determinado**. Por lo que el usuario deberá volver a *loguearse* una vez que el token expire y solicitar otro.

¿Cómo funciona el ciclo de vida de un JWT?



Los pasos son los siguientes:

1. El usuario requiere de una autenticación tradicional con el servidor, es decir, usuario y contraseña.
2. El servidor validará que los datos introducidos sean correctos(en la Base de Datos) y generará un token JWT con una clave.
3. Se puede utilizar clave pública y clave privada o bien un clave secreta.
4. El servidor enviará el token al usuario y este lo tendrá que almacenar de alguna forma.
5. Una vez con el token, cuando el usuario realiza una petición al servidor, envía en **el header** el token previamente generado. La aplicación cliente, hará peticiones solicitando recursos, siempre con ese token JWT dentro de un encabezado, que será del **tipo Authorization: Bearer XXXXXXXX**, siendo **Bearer** el tipo de prefijo seguido de todo el contenido del token.
6. El servidor validará que el token sea correcto, descriptándolo mediante una clave que utilizó para encriptarlo.
7. Si el token es correcto, entonces el servidor retornará los datos/recursos solicitados por el usuario.

¿Cómo implementar un JWT?

Podemos crear nuestras propias librerías, con las clases y métodos para realizar todas las operaciones, pero no es lo más conveniente en un ambiente de producción.

Existen múltiples librerías que generan y validan tokens JWT (JSON Web Token) respetando un estándar que está recogido en el documento RFC 7519. La librería elegida dependerá del lenguaje que utilicemos.

<https://github.com/firebase/php-jwt>

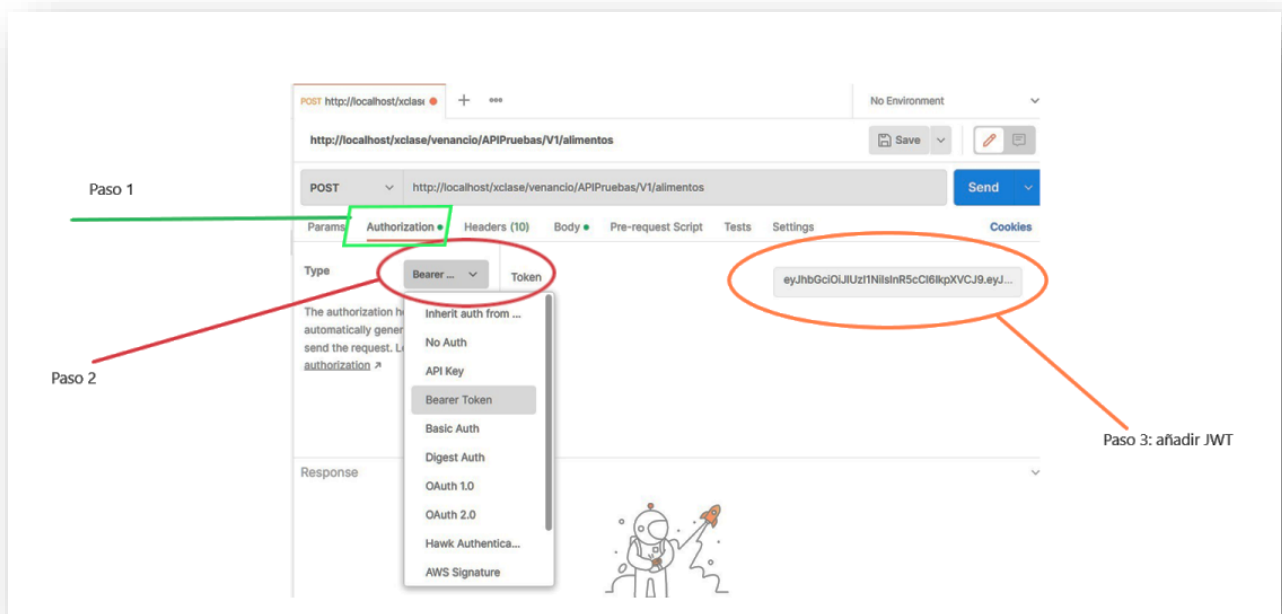
Nosotros vamos a utilizar la librería `firebase/php-jwt` para PHP que instalaremos utilizando **Composer** con el siguiente comando:

composer require firebase/php-jwt

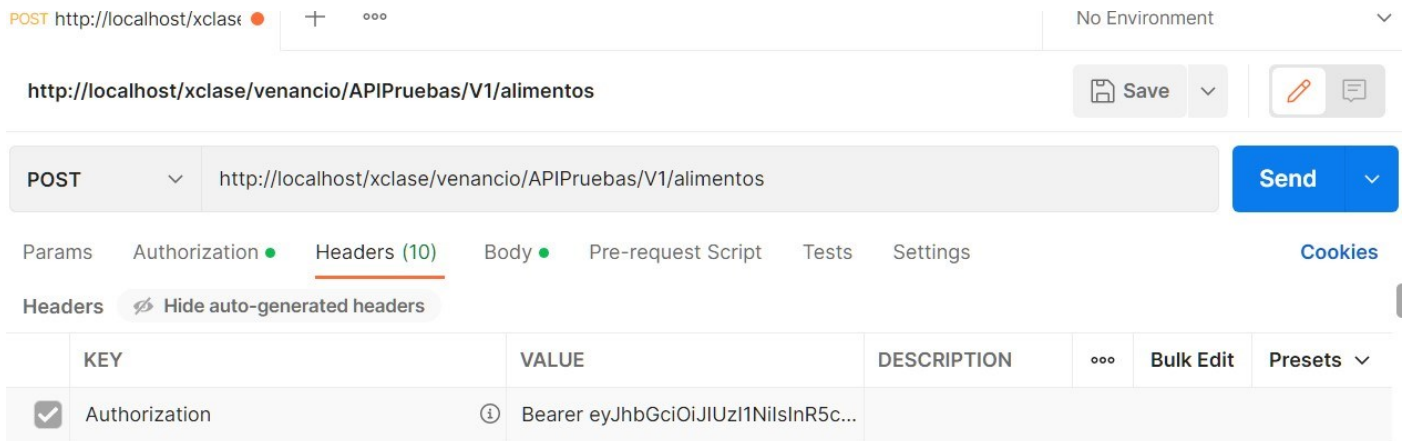
Recordad que hay que incluir el **archivo autoload.php** y el espacio de nombre dónde se encuentran las librerías:

```
<?php
use Firebase\JWT\JWT;
use Firebase\JWT\Key;
class JWTController
{
    private $privateKey;
    private $publicKey;
    public function __construct()
    {
        $ruta1 = dirname(__DIR__, 3) . '/keys/private_key.pem';
        $ruta2 = dirname(__DIR__, 3) . '/keys/public_key.pem';
        $this->privateKey = file_get_contents($ruta1);
        $this->publicKey = file_get_contents($ruta2);
    }
    public function generarToken($usuario)
    {
        $horaActual = time();
        $tiempoExpira = $horaActual + 3600; // JWT válido por 1 hora
        $payload = [
            'iat' => $horaActual, // Hora en la que se emitió el token
            'exp' => $tiempoExpira, // Tiempo en el que expira el token
            'idusuario' => $usuario['id'],
            'email' => $usuario['email'],
            'rol' => $usuario['rol']
        ];
        $jwt = JWT::encode($payload, $this->privateKey, 'ES256');
        return $jwt;
    }
    public function validarToken($jwt)
    {
        $arrayDatosToken = [];
        $decoded = JWT::decode($jwt, new Key($this->publicKey, 'ES256'));
        $arrayDatosToken = (array) $decoded;
        return $arrayDatosToken;
    }
}
```

Cómo enviar datos un token JWT desde Postman



Postman añadirá la siguiente cabecera cuando realicemos el envío.



MUY IMPORTANTE: CAMBIOS en el fichero .htaccess

Varios módulos de Apache pueden eliminar la cabecera de **Authorization** enviada por el cliente, entonces en PHP no tendremos forma de acceder al token JWT enviado desde el cliente.

La solución es pasar la cabecera **Authorization** directamente PHP a través de las variables de entorno de Apache y así aparecerá recogida en `$_SERVER`, para ello vamos a crear un archivo .htaccess con la siguiente línea:

```
SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1
```

Módulo Apache **mod_setenvif** : Permite establecer variables de entorno en función de las características de la petición.

Ejemplo de lo que se debe recibir

```
$contenidoAuth = 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
```

```
$contenidoAuth = $_SERVER['HTTP_AUTHORIZATION'] ?? '';  
// Verificar si el encabezado Authorization contiene el prefijo Bearer  
if (strpos($contenidoAuth, 'Bearer ') === 0) {  
    // Extraer el token JWT eliminando el prefijo Bearer y el espacio  
    $token = substr($contenidoAuth, 7);  
} else {  
    // Si no contiene el prefijo Bearer, enviar una respuesta de error  
    self::$datosConsulta = ["error" => "Encabezado Authorization no válido"];  
    self::enviarRespuesta(400);  
    return;  
}
```

Depurador online de JWT

Se puede utilizar un depurador online para decodificar el token y visualizar su contenido.

Página oficial de JWT para realizar pruebas y decodificar tokens:

<https://jwt.io/>